

Project Report

Introduction

This project is a 2D computer graphics simulation of a moving steam train, developed to demonstrate fundamental animation, user interaction, and shape rendering techniques. Set against a vibrant landscape with a dynamic sky gradient, the simulation features a train that continuously traverses the screen with physically synchronized wheel rotation and a dynamic smoke particle system. The core uniqueness of this project lies in its strict rendering constraints: every element on the screen—from solid circles and rectangles to complex shapes like the cowcatcher—is constructed entirely from fundamental 1D line segments, showcasing the power of basic mathematics in computer graphics.

Problem Statement

What problem exists:

Modern graphics development often relies heavily on high-level libraries, pre-built polygon functions, or external game engines to render solid shapes and handle physics. Relying solely on these tools can abstract away the underlying mathematics of rendering, leaving students and developers with a gap in understanding how primitive shapes are mathematically constructed, filled, and animated at the lowest level.

What your project addresses:

This project addresses this gap by imposing a strict constraint: constructing an entire animated, interactive scene using only basic line primitives (`GL_LINES`) and integer-based vertices (`glVertex2i`). It demonstrates how to manually implement algorithms for solid shape filling (like horizontal line scanning for circles and rectangles), linear interpolation for gradients,

and basic physics calculations for particle systems, effectively building a complex scene from fundamental graphical building blocks.

4. Objectives

- **Simulate Realistic Movement:** To create a smooth, continuous 2D animation of a train moving across the screen with mathematically synchronized wheel rotation.
- **Develop a Particle System:** To implement a dynamic, alpha-blended smoke effect that simulates physics (rising, expanding, and fading over time).
- **Understand Primitive Constraints:** To construct all solid and hollow visual elements (circles, polygons, gradients) strictly using the fundamental GL_LINES primitive.
- **Implement User Interaction:** To allow real-time control over the simulation environment via keyboard inputs, specifically allowing the user to dynamically increase or decrease the train's speed.
- **Optimize Performance:** To maintain an efficient, double-buffered animation loop running at a steady frame rate (~60 FPS) without memory leaks.

5. Algorithms Used

To construct the scene under the strict GL_LINES constraint, several mathematical algorithms and concepts were utilized:

- **Circle Rasterization (Horizontal Line Scanning):** To draw filled circles without polygons, the Pythagorean theorem is utilized. By iterating through the y-axis, the algorithm calculates the corresponding x-axis offset to draw horizontal lines between the two calculated boundaries.
- **Trigonometric Rotation:** Wheel spokes and circular outlines are calculated using standard sine and cosine functions. The endpoints of the

lines are updated continuously based on a rotating angle variable that scales proportionally with the train's movement speed.

- **Linear Interpolation (Lerp):** The sky background is drawn using horizontal lines where the RGB color values are linearly interpolated based on the vertical position (y-coordinate) to create a smooth, seamless gradient transition from dark blue to light blue.
- **Particle System Physics:** The smoke animation relies on a continuous update loop where each particle's variables (x and y velocity, radius, and alpha transparency) are modified mathematically per frame. A randomization algorithm is applied to the spawn position and drift to create an organic, non-uniform smoke plume.

6. Implementation

Programming Language Used:

- **C++:** Chosen for its speed, efficiency, and robust standard library. The standard template library (STL), specifically `std::vector`, is used to dynamically manage the memory of the smoke particle system, allowing particles to be created and destroyed efficiently as they fade out.

Graphics Libraries/Tools:

- **OpenGL (Open Graphics Library):** The core API used for rendering 2D vector graphics. Specifically, the project heavily relies on the `GL_LINES` primitive, `GL_BLEND` for transparency, and matrix transformations for component-based animation.
- **GLUT (OpenGL Utility Toolkit):** Used for window system independence. GLUT handles the creation of the 800x600 display window, capturing keyboard events for user interaction, and managing the timer-based animation loop. Double buffering (`GLUT_DOUBLE`) is implemented to completely eliminate visual flickering.

Code:

```
1#include <GL/glut.h>
2#include <bits/stdc++.h>
3using namespace std;
```

```

4
5 float trainX = -500.0f;    // train horizontal position
6 float wheelAngle = 0.0f;   // wheel rotation angle (degrees)
7 float trainSpeed = 1.5f;   // train movement speed
8 struct SmokeParticle {
9     float x, y, alpha, size, speed;
10 };
11 // Solid filled circle using horizontal line scanning
12 vector<SmokeParticle> smoke;
13 void drawCircle(int cx, int cy, int r) {
14     glBegin(GL_LINES);
15     for (int dy = -r; dy <= r; ++dy) {
16         int dx = round(sqrt(r * r - dy * dy));
17         glVertex2i(cx - dx, cy + dy);
18         glVertex2i(cx + dx, cy + dy);
19     }
20     glEnd();
21 }
22 // Hollow circle outline using GL_LINES segments
23 void drawCircleOutline(int cx, int cy, int r, int segs = 64) {
24     glBegin(GL_LINES);
25     int prevX = cx + r;
26     int prevY = cy;
27     for (int i = 1; i <= segs; ++i) {
28         float a = 2.0f * M_PI * i / segs;
29         int x = cx + round(r * cos(a));
30         int y = cy + round(r * sin(a));
31         glVertex2i(prevX, prevY);
32         glVertex2i(x, y);
33         prevX = x; prevY = y;
34     }
35     glEnd();
36 }
37 // Solid filled rectangle using horizontal line scanning
38 void drawRect(int x, int y, int w, int h) {
39     glBegin(GL_LINES);
40     for (int i = 0; i < h; ++i) {
41         glVertex2i(x, y + i);
42         glVertex2i(x + w, y + i);
43     }
44     glEnd();
45 }
46 // Sky gradient filled with horizontal GL_LINES
47 void drawSky() {
48     glBegin(GL_LINES);
49     for (int y = 200; y <= 600; ++y) {
50         float t = (float)(y - 200) / 400.0f;
51         // Interpolate from bottom color to top color
52         float r = 0.87f + t * (0.53f - 0.87f);
53         float g = 0.95f + t * (0.81f - 0.95f);

```

```

54  float b = 1.00f + t * (0.98f - 1.00f);
55  glColor3f(r, g, b);
56  glVertex2i(0, y);
57  glVertex2i(800, y);
58  }
59  glEnd();
60  }
61  // Ground
62  void drawGround() {
63  glColor3f(0.42f, 0.56f, 0.26f);
64  drawRect(0, 0, 800, 215);
65  glColor3f(0.48f, 0.65f, 0.30f);
66  drawRect(0, 210, 800, 12);
67  }
68  // Railway tracks
69  void drawTracks() {
70  glColor3f(0.35f, 0.22f, 0.08f);
71  for (int x = 0; x <= 800; x += 40) {
72  drawRect(x, 192, 26, 10);
73  }
74  glColor3f(0.60f, 0.60f, 0.65f);
75  drawRect(0, 200, 800, 6);
76  drawRect(0, 187, 800, 6);
77  glColor3f(0.85f, 0.85f, 0.90f);
78  drawRect(0, 203, 800, 2);
79  drawRect(0, 190, 800, 2);
80  }
81  // Wheel
82  void drawWheel(int cx, int cy, int r) {
83  glColor3f(0.15f, 0.15f, 0.15f);
84  drawCircle(cx, cy, r);
85  glColor3f(0.80f, 0.30f, 0.10f);
86  drawCircle(cx, cy, round(r * 0.55f));
87  glColor3f(0.90f, 0.50f, 0.20f);
88  glLineWidth(2.5f);
89  glPushMatrix();
90  glTranslatef(cx, cy, 0);
91  glRotatef(wheelAngle, 0, 0, 1);
92  glBegin(GL_LINES);
93  for (int i = 0; i < 6; ++i) {
94  float a = 2.0f * M_PI * i / 6;
95  glVertex2i(0, 0);
96  glVertex2i(round((r - 2) * cos(a)), round((r - 2) * sin(a)));
97  }
98  glEnd();
99  glPopMatrix();
100 glColor3f(0.20f, 0.20f, 0.20f);
101 drawCircle(cx, cy, round(r * 0.12f));
102 glColor3f(0.30f, 0.30f, 0.30f);
103 glLineWidth(1.5f);

```

```

104 drawCircleOutline(cx, cy, r);
105 }
106 // Train
107 void drawTrain(int tx) {
108     int tenderX = tx - 155;
109     glColor3f(0.20f, 0.20f, 0.22f);
110     drawRect(tenderX, 215, 130, 55);
111     glColor3f(0.15f, 0.15f, 0.17f);
112     drawRect(tenderX + 5, 260, 120, 12);
113     glColor3f(0.12f, 0.12f, 0.12f);
114     drawRect(tenderX + 10, 265, 110, 8);
115     drawWheel(tenderX + 25, 205, 18);
116     drawWheel(tenderX + 75, 205, 18);
117     drawWheel(tenderX + 115, 205, 18);
118     glColor3f(0.50f, 0.50f, 0.50f);
119     drawRect(tx - 25, 220, 28, 8);
120     glColor3f(0.85f, 0.20f, 0.12f);
121     drawRect(tx, 220, 170, 60);
122     glColor3f(0.80f, 0.18f, 0.10f);
123     drawCircle(tx + 50, 283, 18);
124     glColor3f(0.25f, 0.22f, 0.20f);
125     drawRect(tx + 115, 230, 60, 70);
126     glColor3f(0.18f, 0.15f, 0.13f);
127     drawRect(tx + 110, 298, 70, 10);
128     glColor3f(0.55f, 0.85f, 0.95f);
129     drawRect(tx + 122, 258, 20, 22);
130     drawRect(tx + 150, 258, 20, 22);
131
132     // Window frames (Using GL_LINES)
133     glColor3f(0.60f, 0.60f, 0.60f);
134     glLineWidth(1.5f);
135     glBegin(GL_LINES);
136     glVertex2i(tx + 122, 258); glVertex2i(tx + 142, 258);
137     glVertex2i(tx + 142, 258); glVertex2i(tx + 142, 280);
138     glVertex2i(tx + 142, 280); glVertex2i(tx + 122, 280);
139     glVertex2i(tx + 122, 280); glVertex2i(tx + 122, 258);
140     glVertex2i(tx + 150, 258); glVertex2i(tx + 170, 258);
141     glVertex2i(tx + 170, 258); glVertex2i(tx + 170, 280);
142     glVertex2i(tx + 170, 280); glVertex2i(tx + 150, 280);
143     glVertex2i(tx + 150, 280); glVertex2i(tx + 150, 258);
144     glEnd();
145     glColor3f(0.20f, 0.18f, 0.16f);
146     drawRect(tx - 2, 222, 24, 55);
147     drawCircle(tx + 10, 250, 14);
148     glColor3f(0.50f, 0.50f, 0.50f);
149     drawCircle(tx + 10, 250, 6);
150     glColor3f(0.70f, 0.15f, 0.08f);
151     for (int i = 0; i < 3; ++i)
152         drawRect(tx + 30 + i * 28, 220, 5, 60);
153

```

```

154 glColor3f(0.30f, 0.30f, 0.30f);
155 drawRect(tx - 5, 217, 185, 8);
156 glColor3f(0.50f, 0.50f, 0.55f);
157 glBegin(GL_LINES);
158 for (int y = 207; y <= 217; ++y) {
159     int dx = round(30.0f * (217 - y) / 10.0f);
160     glVertex2i(tx - dx, y);
161     glVertex2i(tx, y);
162 }
163 for (int y = 207; y <= 210; ++y) {
164     glVertex2i(tx - 30, y);
165     glVertex2i(tx, y);
166 }
167 glEnd();
168 glColor3f(1.0f, 0.95f, 0.60f);
169 drawCircle(tx + 2, 265, 7);
170 glColor3f(1.0f, 1.0f, 0.80f);
171 drawCircle(tx + 2, 265, 3);
172 glColor3f(0.18f, 0.15f, 0.13f);
173 drawRect(tx + 15, 280, 22, 35);
174 drawRect(tx + 10, 312, 32, 8);
175 drawWheel(tx + 30, 205, 22);
176 drawWheel(tx + 80, 205, 22);
177 drawWheel(tx + 130, 205, 22);
178 drawWheel(tx - 10, 207, 14);
179 drawWheel(tx + 162, 207, 14);
180 glColor3f(0.70f, 0.70f, 0.72f);
181 glLineWidth(4.0f);
182 glBegin(GL_LINES);
183 glVertex2i(tx + 30, 205); glVertex2i(tx + 80, 205);
184 glVertex2i(tx + 80, 205); glVertex2i(tx + 130, 205);
185 glEnd();
186 glColor3f(0.60f, 0.60f, 0.62f);
187 drawRect(tx + 5, 212, 30, 5);
188 glLineWidth(2.0f);
189 glBegin(GL_LINES);
190 glVertex2i(tx + 35, 214); glVertex2i(tx + 30, 205);
191 glEnd();
192 }
193 // Smoke
194 void spawnSmoke(float baseX, float baseY) {
195     if (smoke.size() < 40) {
196         SmokeParticle p;
197         p.x = baseX + (rand() % 10 - 5);
198         p.y = baseY;
199         p.alpha = 0.75f + 0.25f * (rand() % 100) / 100.0f;
200         p.size = 10.0f + rand() % 12;
201         p.speed = 0.6f + 0.4f * (rand() % 100) / 100.0f;
202         smoke.push_back(p);
203     }

```

```

204 }
205 void drawSmoke() {
206     for (auto &p : smoke) {
207         glColor4f(0.55f, 0.55f, 0.55f, p.alpha);
208         drawCircle(round(p.x), round(p.y), round(p.size));
209     }
210 }
211 void updateSmoke(float stackX, float stackY) {
212     for (auto &p : smoke) {
213         p.y += p.speed * 1.2f;
214         p.x += (rand() % 3 - 1) * 0.3f;
215         p.size += 0.25f;
216         p.alpha -= 0.008f;
217     }
218     smoke.erase(
219         remove_if(smoke.begin(), smoke.end(), [](const SmokeParticle &p) { return p.alpha <= 0;
220             }),
221         smoke.end()
222     );
223     spawnSmoke(stackX, stackY);
224 }
225 // Keyboard Control
226 void keyboard(unsigned char key, int x, int y) {
227     if (key == 'u' || key == 'U') {
228         trainSpeed += 0.5f;    // Increase speed
229     } else if (key == 'd' || key == 'D') {
230         trainSpeed -= 0.5f;    // Decrease speed
231     } if (trainSpeed < 0.0f) {
232         trainSpeed = 0.0f;    // Prevent train from going backwards
233     }
234 }
235 void display() {
236     glClear(GL_COLOR_BUFFER_BIT);
237     drawSky();
238     drawGround();
239     drawTracks();
240
241     glEnable(GL_BLEND);
242     glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
243     drawSmoke();
244     glDisable(GL_BLEND);
245
246     drawTrain((int)trainX);
247
248     glutSwapBuffers();
249 }
250 // Timer / Animation
251 void timer(int) {
252     trainX += trainSpeed;

```



```

253 if (trainX > 900.0f) trainX = -320.0f;
254 wheelAngle -= (trainSpeed * 2.33f);
255 if (wheelAngle < -360.0f) wheelAngle += 360.0f;
256 updateSmoke(trainX + 26, 318);
257 glutPostRedisplay();
258 glutTimerFunc(16, timer, 0);
259 }
260 int main(int argc, char **argv) {
261     srand(time(nullptr));
262
263     glutInit(&argc, argv);
264     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
265     glutInitWindowSize(800, 600);
266     glutInitWindowPosition(100, 100);
267     glutCreateWindow("Train Movement Animation");
268     glClearColor(0.53f, 0.81f, 0.98f, 1.0f);
269     glMatrixMode(GL_PROJECTION);
270     glLoadIdentity();
271     gluOrtho2D(0, 800, 0, 600);
272     glMatrixMode(GL_MODELVIEW);
273     glLoadIdentity();
274     glutDisplayFunc(display);
275     glutKeyboardFunc(keyboard);
276     glutTimerFunc(16, timer, 0);
277     glutMainLoop();
278     return 0;
279 }

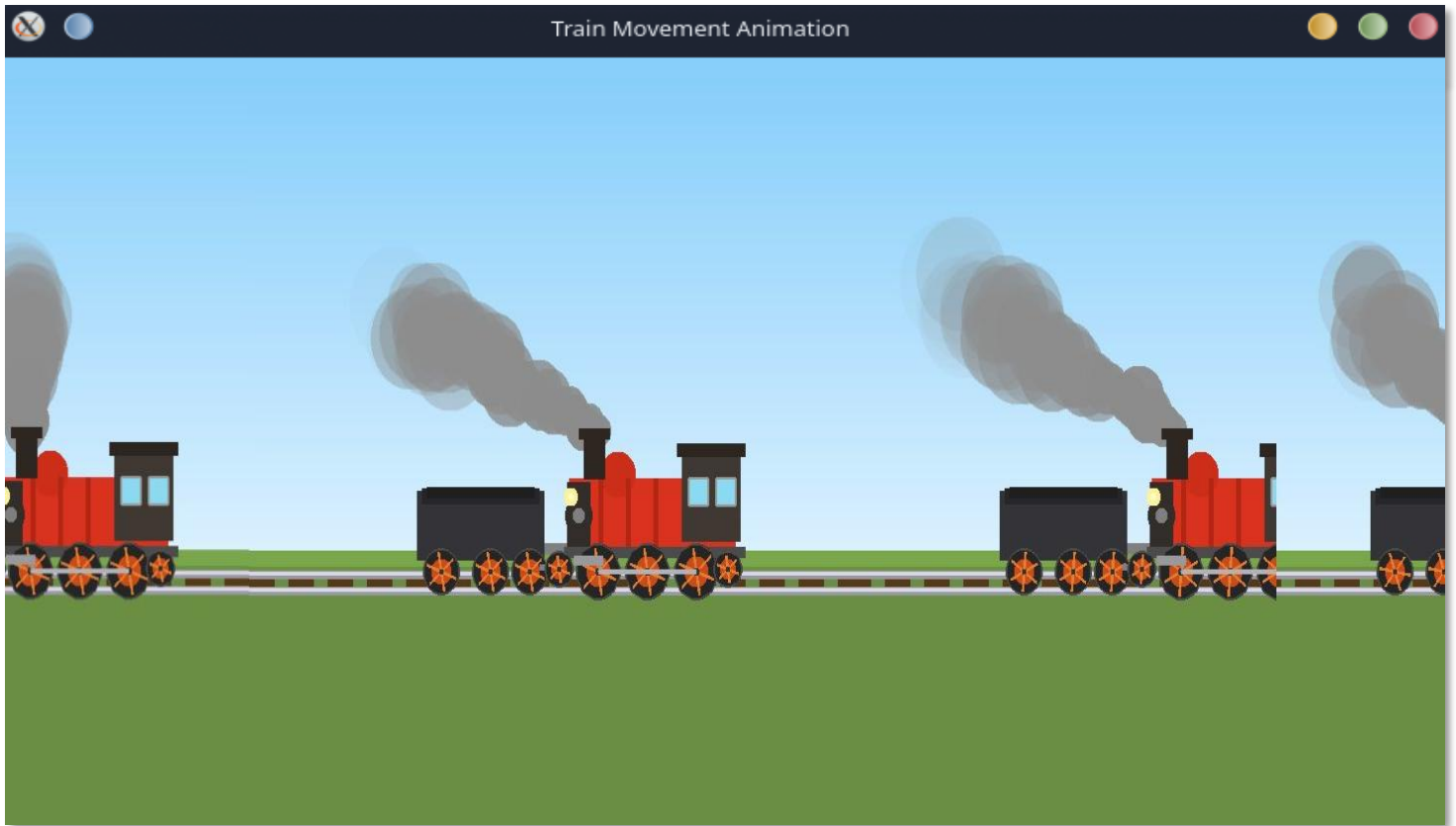
```

7. Results/Outputs:

Upon execution, the program successfully launches an 800x600 interactive window displaying a vividly colored, side-scrolling 2D environment. The train continuously loops across the screen, moving left to right. The visual outputs include:

- **Solid Shape Rendering:** Despite being built from lines, all train parts, tracks, and scenery appear as perfectly solid, colored objects.
- **Synchronized Animation:** The rotation of the train's wheels directly matches the horizontal velocity of the train body, creating a realistic sense of traction.
- **Dynamic Particles:** Semi-transparent smoke realistically puffs from the chimney, expanding and fading as it drifts upward and backward.

- **Interaction:** Pressing the 'u' key visibly accelerates both the train's forward movement and wheel rotation. Pressing the 'd' key dynamically decelerates the train until it comes to a complete halt, demonstrating real-time variable manipulation.



8. Conclusion

The Train Movement Animation project successfully proves that complex visual scenes and dynamic animations can be built from the ground up using minimal graphical primitives. By restricting the rendering to simple line segments, the project reinforced core computer graphics concepts such as algorithmic shape generation, trigonometric animation, particle physics, and state-driven event loops. The final product is a highly optimized, interactive, and visually engaging simulation that serves as a strong foundation for understanding lower-level computer graphics programming.